

OBSERVABILITY SRE PROJECT

AWS Kubernetes Lab

End-to-end observability stack for Kubernetes microservices deployed on AWS EC2. This project demonstrates how to build a complete monitoring, logging, and tracing pipeline using industry-standard open-source tools.

WHAT THIS PROJECT COVERS:

- Real Kubernetes cluster on AWS EC2 (t3.xlarge, us-east-2)
- 30+ microservices from OpenTelemetry Demo
- Full monitoring stack: Prometheus, Grafana, Loki, Tempo
- Golden Signals dashboard + SLO definitions

Aleksandr Dergachev | Yurik Mnatsakanian | Cloud & DevOps Engineers



Kubernetes



Prometheus



Grafana



AWS EC2



Loki



Tempo



k6



OpenTelemetry

PROJECT GOAL



Application Monitoring



Centralized Logging



Distributed Tracing



System Health Visibility



Reliability Metrics



Load Testing

EXPLANATION

What we built:

- A complete observability system for a live Kubernetes application on AWS EC2.

Application Monitoring

- Track what every service is doing in real time using Prometheus metrics.

Centralized Logging

- All pod logs collected in one place with Loki, searchable from Grafana.

Distributed Tracing

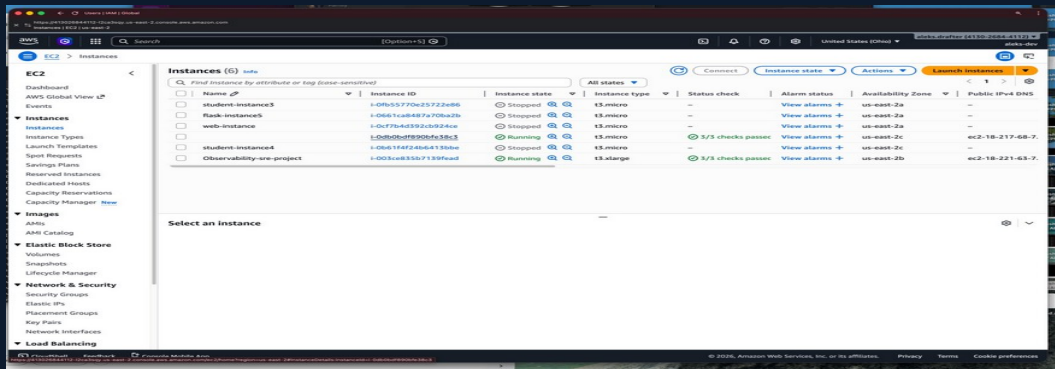
- Follow one user request across multiple services using Tempo.

Load Testing

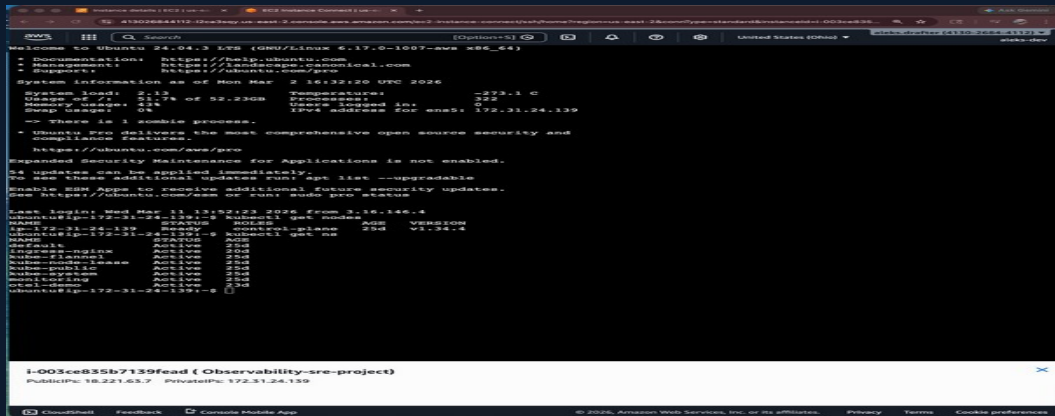
- k6 validated performance: 9-10ms avg latency, zero failed requests.

INFRASTRUCTURE — AWS EC2 KUBERNETES CLUSTER

AWS EC2 Console — Observability-sre-project Running



kubectl get nodes + get ns — v1.34.4 Ready, 8 namespaces



EXPLANATION

Why these choices:

- **t3.xlarge (4 CPU, 16GB RAM)**
- Enough power to run 30+ microservices plus the full monitoring stack simultaneously.
- **kubeadm (not EKS)**
- Self-managed cluster gives full control over every component — exactly like a real production environment.
- **Flannel CNI**
- Provides pod-to-pod networking across the cluster using an overlay network.
- Node: ip-172-31-24-139
- Status: Ready · 8 namespaces active

APPLICATION LAYER — OPENTELEMETRY DEMO

OpenTelemetry Demo Microservices

Namespace: otel-demo · 30+ pods running

kubectl get pods -A — все поды Running (live cluster)

```
ubuntu@ip-172-31-24-139:~$ kubectl get pods -A
NAME                                NAMESPACE                                STATUS    RESTARTS    AGE
kube-flannel                        kube-system                              Running   0            1m
coredns-66bc5c9577-73p88            kube-system                              Running   0            1m
coredns-66bc5c9577-nw79g            kube-system                              Running   0            1m
etcd-ip-172-31-24-139               kube-system                              Running   0            1m
kubernetes-applier-172-31-24-139     kube-system                              Running   0            1m
kubernetes-controller-manager-172-31-24-139 kube-system                              Running   0            1m
kubernetes-proxy-6115r              kube-system                              Running   0            1m
kubernetes-scheduler-172-31-24-139   kube-system                              Running   0            1m
alertmanager-mon-kube-prometheus-stack-alertmanager-0 monitoring                              Running   0            1m
loki-0                              loki-0                                  Running   0            1m
loki-promtail-9n81g                 loki-0                                  Running   0            1m
mon-grafana-6474459f8b-vh88j        monitoring                              Running   0            1m
mon-kube-prometheus-stack-operator-7b5d9f9469-she9d monitoring                              Running   0            1m
mon-kube-prometheus-stack-operator-7cd5f45bf9-4gr75 monitoring                              Running   0            1m
mon-kube-state-metrics-7f88894956-btdbk monitoring                              Running   0            1m
mon-prometheus-node-exporter-q1jgm   monitoring                              Running   0            1m
prometheus-mon-kube-prometheus-stack-prometheus-0 monitoring                              Running   0            1m
tempo-0                             monitoring                              Running   0            1m
otel-demo-accounting-76dc9dc54-b4s2z otel-demo                              Running   0            1m
otel-demo-ad-667f9497cd-8x47v        otel-demo                              Running   0            1m
otel-demo-cart-86677d8cc4-qm1x6       otel-demo                              Running   0            1m
otel-demo-checkout-749df7be8c-9kfmf    otel-demo                              Running   0            1m
otel-demo-currency-7b456dbf59-79asn    otel-demo                              Running   0            1m
otel-demo-email-78ddc489b9-8g6dv       otel-demo                              Running   0            1m
otel-demo-flagd-84ff977b59-q8xb2       otel-demo                              Running   0            1m
otel-demo-fr-aud-detect-1on-4cbb8969b-mg9dd otel-demo                              Running   0            1m
otel-demo-frontend-786d8489f5-hjtwc    otel-demo                              Running   0            1m
otel-demo-frontend-proxy-707d4d57fb-lqvrw otel-demo                              Running   0            1m
otel-demo-grafana-659bc9bdbc-7xbyvs     otel-demo                              Running   0            1m
otel-demo-image-provider-6ff8cd697-z46s5 otel-demo                              Running   0            1m
otel-demo-leanser-5b58df596-9wvxp       otel-demo                              Running   0            1m
otel-demo-kafka-57f5c79b96-dg5vm       otel-demo                              Running   0            1m
otel-demo-llm-749dc4d84b-gp5fz         otel-demo                              Running   0            1m
otel-demo-load-generator-767d49b7d5-5xqgz otel-demo                              Running   0            1m
otel-demo-opensearch-0                otel-demo                              Running   0            1m
otel-demo-otel-collector-agent-j5146   otel-demo                              Running   0            1m
otel-demo-payment-678ff67d7-2zmjl      otel-demo                              Running   0            1m
otel-demo-postgresql-546448cc67-186n6  otel-demo                              Running   0            1m
otel-demo-product-catalog-75b49dc4bf-fa4k2 otel-demo                              Running   0            1m
otel-demo-product-reviewer-6cd998b87-dsmfz otel-demo                              Running   0            1m
otel-demo-prometheus-c97c9fb9f-6b9g9   otel-demo                              Running   0            1m
otel-demo-quote-74c95b79c8-hqwrft      otel-demo                              Running   0            1m
otel-demo-recommendation-67dbff6bd-n5xqv otel-demo                              Running   0            1m
otel-demo-shipment-bf97f8899-7bxxz     otel-demo                              Running   0            1m
otel-demo-travelkey-cart-76f8f6f939-axl7f otel-demo                              Running   0            1m
otel-demo-wgettest                    otel-demo                              Completed 0            1m
```

i-003ce835b7139fead (Observability-sre-project)

PublicIPs: 18.221.63.7 PrivateIPs: 172.31.24.139

EXPLANATION

What is this application?

- The OpenTelemetry Demo is an official open-source e-commerce app with 30+ microservices, designed specifically to generate realistic telemetry data.

Why ClusterIP only?

- All services use ClusterIP — they are internal only and cannot be accessed directly from the internet. This is a security best practice.

Installation (Helm):

- kubectl create namespace otel-demo
- helm install otel-demo open-telemetry/
- opentelemetry-demo -n otel-demo
- After install: verify all pods show
- STATUS = Running before testing.

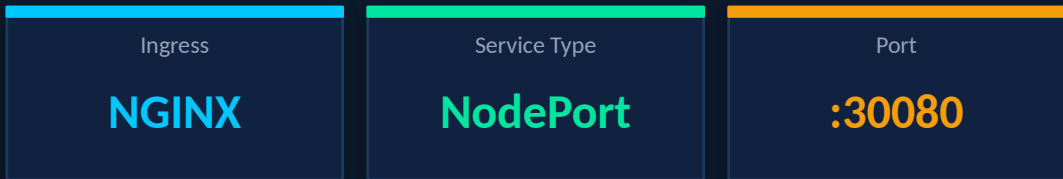
kubectl get pods -n monitoring — все поды Running (live)



- The OpenTelemetry Collector runs inside otel-demo and acts as the central telemetry router.

- i-003ce835b7139fead (Observability-sre-project)**
PublicIPs: 18.221.63.7 PrivateIPs: 172.31.24.139

EXTERNAL ACCESS & SECURITY MODEL



TRAFFIC FLOW



SECURITY DECISIONS

-  Internal services = ClusterIP only (not reachable from internet)
-  Single ingress surface = smaller attack area
-  Monitoring stack not publicly exposed

EXPLANATION

Why single entry point?

- Having one external entry point (NGINX Ingress) is a security best practice. All traffic is inspected and routed from one place.

What is NodePort?

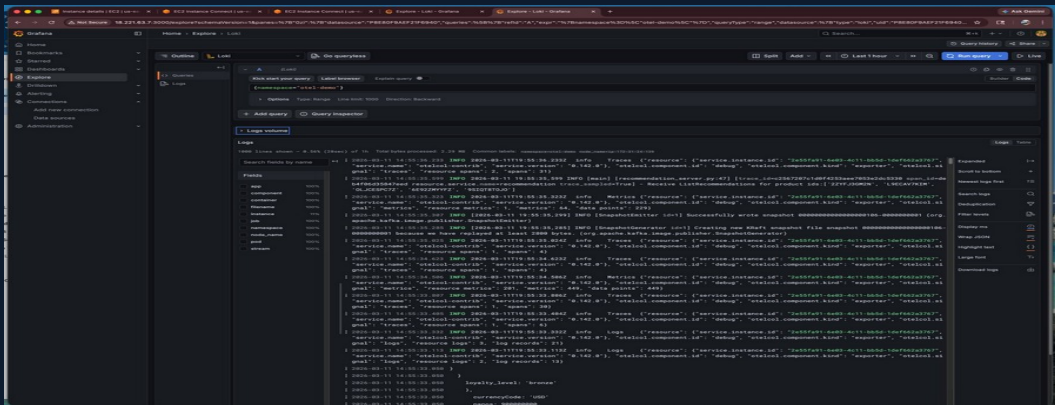
- NodePort opens one port (30080) on the EC2 instance. The cloud firewall allows only that port from outside. Everything else is blocked.

For production environment:

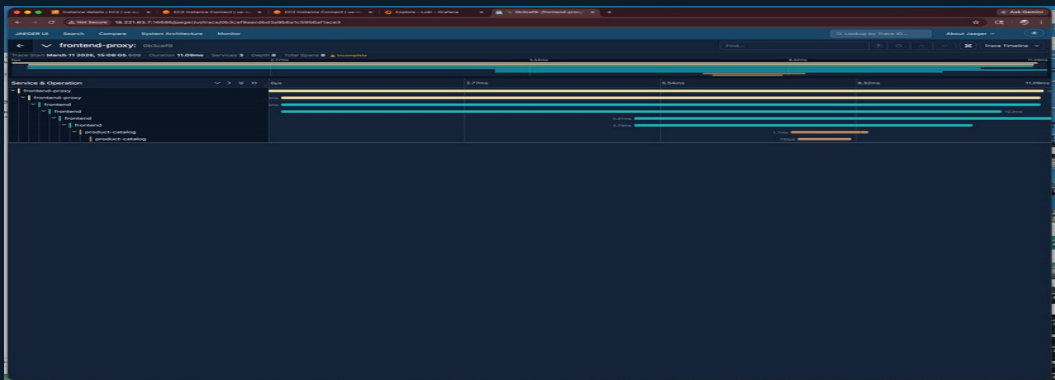
- Replace NodePort → LoadBalancer
- Add HTTPS / TLS certificate
- Add real domain name
- Enable RBAC access control
- Restrict Grafana to internal VPN

OBSERVABILITY VALIDATION — ALL 3 SIGNALS CONFIRMED

Grafana Loki — {namespace=otel-demo} — 1000 lines, 2.29 MB



Jaeger UI — frontend-proxy trace: 11.09ms, 3 services, 8 spans



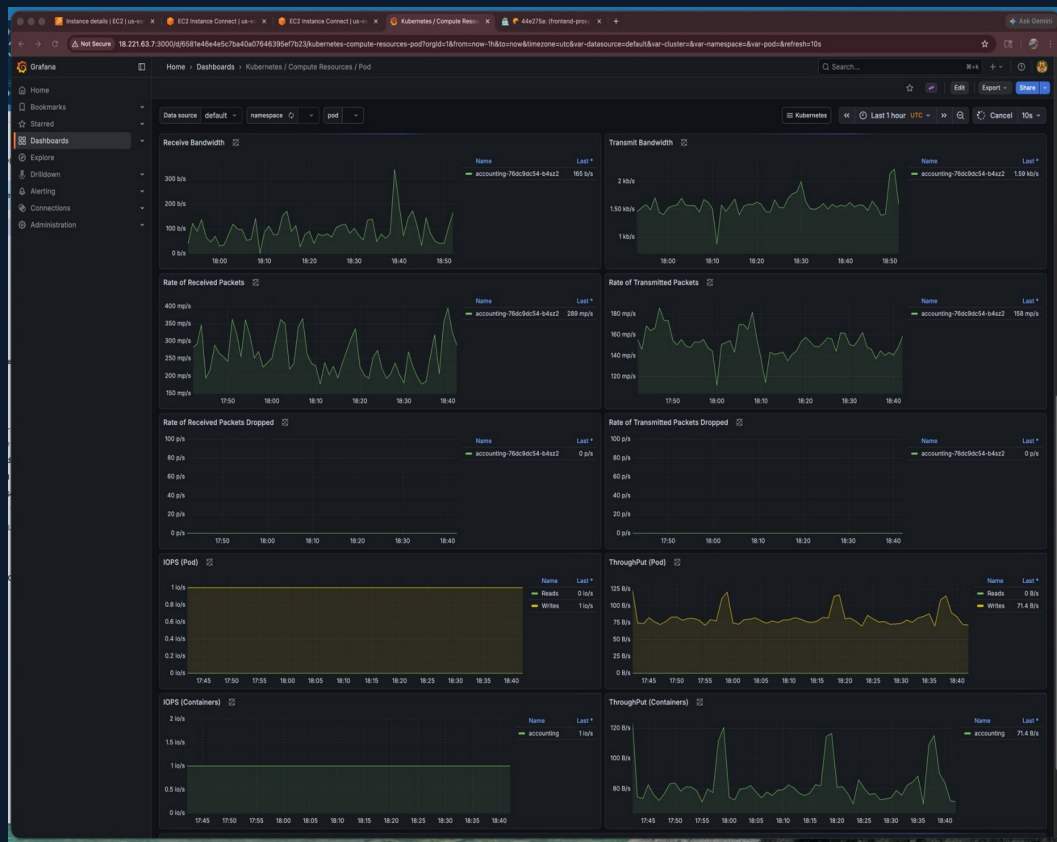
EXPLANATION

How we validated:

- Each of the three observability signals was verified manually after deployment.
- Metrics validation:**
 - Opened Grafana dashboards and verified that request rate, error rate, and latency charts were populated with real data from Prometheus.
- Log validation:**
 - Used Grafana Explore → Loki datasource. Queried the otel-demo namespace and confirmed logs flowing from multiple services.
- Trace validation:**
 - Triggered a real checkout in the app, then found the trace in Grafana Explore → Tempo. Confirmed 3 services, 8 spans, 11ms total.

GOLDEN SIGNALS DASHBOARD — GRAFANA

Live Grafana — Kubernetes / Compute Resources / Pod dashboard



EXPLANATION

What are Golden Signals?

- A methodology from Google's SRE book. These 4 metrics are the minimum you need to understand if your service is healthy.
- **Traffic**
 - How busy is the system? High traffic is normal but sudden spikes need attention.
- **Errors**
 - Are things failing? Any 5xx errors mean users are experiencing problems.
- **Latency**
 - How slow are responses? P95 means: 95% of users get a response faster than this number.
- **Saturation**
 - Are we running out of resources? High CPU means the system is under pressure.
- Dashboard stored in `/dashboards/` for version control and easy re-

SERVICE LEVEL OBJECTIVES (SLOS)



Availability

$\geq 99\%$

Service uptime



Achieved ✓



P95 Latency

$< 500\text{ms}$

Response time



9-10ms ✓



Error Rate

$< 1\%$

HTTP 5xx errors



0% ✓

EXPLANATION

What is an SLO?

- An SLO (Service Level Objective) defines what 'good enough' performance looks like. It is a measurable target, not just a goal.
- **Availability $\geq 99\%$**
 - The service must be up and responding at least 99% of the time. Measured from Prometheus uptime metrics.
- **P95 Latency $< 500\text{ms}$**
 - 95% of all user requests must complete in under 500 milliseconds. Our actual result: 9-10ms — 50x better than the target.
- **Error Rate $< 1\%$**
 - Less than 1% of requests can result in errors. Our k6 load test confirmed 0% errors under full load.
- ✓ All 3 SLOs were met and validated through k6 load testing.

LOAD TESTING — K6



k6 — Open Source Load Testing by Grafana Labs

Tests run via kubectl port-forward on frontend-proxy

Baseline Load Test

Normal expected traffic level to establish performance baseline for the system.

Stress Load Test

Traffic pushed beyond normal to find performance limits and breaking points.

RESULTS

9-10ms

Avg Latency

0%

Failed Requests

PASS

All SLOs Met

EXPLANATION

Why load testing matters:

- Observability without load testing is incomplete. We need to verify that the system performs well not just at rest, but under real traffic.
- **Baseline test:**
 - Simulates normal day-to-day traffic. Establishes what 'normal' looks like for latency and error rate.
- **Stress test:**
 - Pushes traffic until the system starts to struggle. Shows where the limits are before they appear in production.
- **Results stored in:**
 - `load-testing/results/`
- 9-10ms avg latency is exceptional — the SLO target was 500ms. The system is 50x faster than required.

LIVE ENVIRONMENT — REAL CLUSTER DATA

t3.xlarge

EC2 Instance

v1.34.4

Kubernetes

30+

Pods (otel-demo)

us-east-2

AWS Region

NAMESPACES

otel-demo 23d

monitoring 25d

ingress-nginx 20d

kube-flannel 25d

kube-system 25d

MONITORING PODS

prometheus 2/2 Running

grafana 3/3 Running

loki-0 1/1 Running

alertmanager 2/2 Running

tempo-0 1/1 Running

Loki: {namespace=otel-demo} → 1000 lines · 2.29 MB · Grafana :3000 via port-forward

EXPLANATION

This is all real — live cluster:

- Every number on this slide comes directly from the running AWS EC2 instance. This is not a demo environment or simulation.
- **t3.xlarge**
 - 4 vCPUs and 16GB RAM. Handles 30+ services + full monitoring stack simultaneously with 43% memory usage.
- **Grafana 3/3 Running**
 - 3 containers in one pod — this is normal for the kube-prometheus-stack Grafana deployment.
- **loki-0 running for 24 days**
 - The cluster has been stable and continuously running — no crashes or restarts in the monitoring stack.
- 2.29 MB of logs processed in one Loki query — confirms active log ingestion from all otel-demo services.

PROJECT SUMMARY

Complete Observability Pipeline for Kubernetes



Metrics monitoring with Prometheus



Centralized logging with Loki



Distributed tracing with Tempo + Jaeger UI



Unified visualization in Grafana



Load testing with k6 — 0% errors, 9-10ms

WHAT THIS PROVES



Real AWS infrastructure built and managed from scratch



Kubernetes deployed with kubeadm — not a managed service



30+ services running and monitored in production style



All 3 observability signals active: metrics, logs, traces



Performance validated — SLOs met under load testing



Documented, version-controlled, reproducible